

Bank Management System

Software Test Plan

Version 1.0 – IEEE 829 Format

1. Introduction

1.1 Purpose

This Test Plan defines the scope, approach, resources, and schedule for testing Version 1.0 of the Bank Management System (BMS). It is written against the Software Requirements Specification (SRS) v1.0 dated 11/09/2026, and follows the IEEE 829 Standard for Software Test Documentation.

1.2 Scope

Testing covers all functional requirements FR-1 through FR-14 and non-functional requirements NFR-1 through NFR-9 defined in the SRS, spanning authentication, customer and account management, transactions, fund transfer, reporting, interest computation, and staff management. Features explicitly out of scope for Version 1.0 (see Section 4.2) are excluded.

1.3 References

- Software Requirements Specification, Bank Management System, Version 1.0, 11/09/2026.
- IEEE Std. 829-2008 / 830-1998, IEEE Standards for Software Test Documentation and SRS.
- Software Engineering course material (UE24CS341A), PES University.

1.4 Document Conventions

Functional requirements are referenced as FR-n and non-functional requirements as NFR-n, consistent with the SRS. Test case identifiers use the prefix TC- (e.g., TC-01). The keyword “shall” is used for mandatory test-plan provisions.

2. Test Plan Overview

Test Plan Identifier: BMS-TP-1.0. This plan is maintained alongside the project's SRS and Design Specification in the shared GitHub repository (Bank-Management-System) and is updated as the implementation progresses.

3. Test Items

The following software items, corresponding to the modules of the C/C++ back-end core and the web front-end, are subject to test:

Test Item	Description
Authentication & Session module	Login, logout, RBAC enforcement, account lockout
Customer Management module	Create/update/deactivate customer records
Account Management module	Open, freeze/unfreeze, close accounts
Transaction module	Deposit and withdrawal processing
Fund Transfer module	Inter-account transfer with atomicity
Reporting module	Balance inquiry, transaction history, account statements
Interest Computation module	Periodic interest calculation and crediting
Staff Management & Dashboards module	Staff account CRUD, role-specific dashboards
C/C++ back-end core	All business logic, validation, and DB access
Web front-end	Presentation and data-entry layer (no business logic)

4. Features to be Tested and Not to be Tested

4.1 Features to be Tested

Requirement(s)	Feature
FR-1, FR-2	Login, logout, RBAC, account lockout
FR-3	Customer record management
FR-4, FR-5	Account opening, freeze/unfreeze/close
FR-6, FR-7	Deposit and withdrawal
FR-8	Fund transfer (atomic, linked entries)
FR-9, FR-10	Balance inquiry and transaction history
FR-11, FR-12	Interest computation and account statements
FR-13, FR-14	Staff management and role dashboards
NFR-1, NFR-2	Performance (transaction and page-load time)
NFR-3	Atomicity / balance-integrity safety requirement
NFR-4, NFR-5	Security: credential hashing, RBAC enforcement, audit trail
NFR-6, NFR-7, NFR-8, NFR-9	Usability, reliability, maintainability, portability

4.2 Features Not to be Tested

The following are explicitly out of scope for Version 1.0 per SRS Section 1.4 and Appendix D, and are therefore not covered by this Test Plan: loan and EMI management; fixed and recurring deposits; cheque processing and clearing; ATM/debit card operations; internet and mobile banking channels; interbank transfers (NEFT/RTGS/UPI); multi-branch and multi-currency operation; joint accounts; two-factor authentication/SSO; and concurrent multi-terminal operation with record-level locking.

5. Approach

5.1 Security Validation Approach

Security-related requirements (NFR-4, NFR-5) are validated by: (a) inspecting stored credentials to confirm passwords/PINs are held as salted hashes and never appear in plain text in the database, logs, or network traffic; (b) verifying that role-based access control is enforced in the C/C++ back-end core on every request, not merely hidden in the front end — attempted access outside a role's permissions (e.g., a Customer requesting another customer's account) must be denied and logged; (c) confirming the audit trail is append-only by attempting to edit or delete an existing entry through the application; and (d) supplying invalid, boundary, and adversarial inputs (negative amounts, non-numeric values, unknown account numbers) to confirm the system fails safely without corrupting data. Representative cases: TC-02, TC-06, TC-15, TC-16, TC-17.

5.2 Functional Testing Approach

Functional test cases are designed using black-box techniques — equivalence partitioning and boundary value analysis — directly from the FR statements in the SRS (e.g., minimum-balance boundaries, daily withdrawal limits, duplicate interest-period checks). Each System Feature in SRS Section 4 maps to at least one test case in Section 14.

5.3 Non-Functional Testing Approach

Performance requirements (NFR-1, NFR-2) are validated by timing transaction completion and page loads under a representative dataset. Reliability (NFR-7) is validated with invalid-input and error-path testing. Usability (NFR-6) is validated through a walkthrough with at least three representative users per role. Portability (NFR-9) is validated by building the back-end core on two platforms and exercising the front end on two browsers.

5.4 Test Levels

- Unit testing — individual C/C++ functions (validation, interest calculation, balance updates), performed by the developer of each module.
- Integration testing — back-end core against the database and the API layer consumed by the front end.
- System testing — end-to-end scenarios through the web UI, covering the test cases in Section 14.
- Acceptance testing — a walkthrough demo against the SRS, conducted with the course instructor/evaluator.

6. Item Pass/Fail Criteria

A test case passes when the actual result matches the expected result stated in Section 14, with no unhandled exception, crash, or unintended data modification. A feature is considered to have passed testing when all of its associated test cases pass. Any deviation is logged as a defect with a severity (Critical/Major/Minor) and re-tested after a fix.

7. Suspension Criteria and Resumption Requirements

Testing of a dependent feature is suspended if a blocking defect prevents further execution (e.g., a broken login blocks all role-scoped test cases) or if the build/test environment is unavailable. Testing resumes once the blocking defect is fixed and verified, or the environment is restored, whichever applies.

8. Test Deliverables

- This Test Plan document, including the test cases in Section 14.
- Test execution log / defect log maintained during system testing.
- A test summary report at the conclusion of testing, before project submission.

9. Environmental Needs

Hardware: standard development laptops (≥ 16 GB RAM). Software: a standards-compliant C/C++ toolchain (GCC/G++ or MSVC), a relational database (as selected by the team), a modern web browser (Chrome and Firefox, for cross-browser checks), Git, and a GitHub repository for version control. Test data: a seeded set of customer, account, and staff records covering active, frozen, and closed states to allow repeatable test execution.

10. Responsibilities

Name	Responsibility
Abhishek Hanamant Kinagi	Test planning, back-end/core test cases, security validation
Adithya Ajeeth	Front-end/UI test cases, usability walkthrough
Aditya M	Transaction and fund-transfer test cases, defect tracking
Aditya Vats	Account/customer management test cases, environment setup

11. Schedule

Test planning and test case design proceed alongside module development. System testing is performed once each feature's implementation is merged to the main branch, with a final regression pass before the project submission deadline.

12. Risks and Contingencies

- Limited time due to academic deadlines — mitigated by prioritizing High-priority FRs (authentication, transactions, transfers) first.
- Integration issues between the C/C++ core and the chosen database/API layer — mitigated by early integration testing rather than leaving it to the end.
- Team member unavailability — mitigated by cross-familiarity across modules and shared ownership of the test case set.

13. Approvals

This Test Plan is approved for execution by the project team and reviewed by the course instructor/evaluator.

Team Lead: _____ Date: _____

Instructor/Evaluator: _____ Date: _____

14. Test Cases

Seventeen test cases (TC-01 to TC-17) are defined, covering the functional requirements across authentication, customer/account management, transactions, transfers, reporting and staff management, together with representative non-functional requirements for performance, safety/atomicity, security and reliability – satisfying the minimum of 7-10 test cases with coverage of both functional and non-functional requirements.

14.1 Requirement Traceability Summary

Requirement ID	Requirement Summary	Covered By
FR-1	User login, RBAC enforcement	TC-01
FR-2	Logout, account lockout after 3 failed attempts	TC-02
FR-3	Customer record create / validation	TC-03, TC-04
FR-4	Account opening, minimum-balance check	TC-05
FR-5	Freeze / unfreeze / close account	TC-06
FR-6	Deposit transaction	TC-07
FR-7	Withdrawal, minimum-balance & limit checks	TC-08
FR-8	Fund transfer, atomicity	TC-09, TC-10
FR-9 / FR-10	Balance inquiry & transaction history, access scoping	TC-11
FR-11	Interest computation, no double-credit	TC-12
FR-13	Staff account deactivation, historical attribution	TC-13
NFR-1	Transaction performance ($\leq 2s$)	TC-14
NFR-3	Atomicity / no negative or sub-minimum balance	TC-09
NFR-4	Password/PIN hashing, no plain-text exposure	TC-15
NFR-5	Append-only audit trail	TC-16
NFR-7	Graceful handling of invalid input	TC-17

14.2 Detailed Test Cases

FR-prefixed rows are functional test cases; NFR-prefixed rows are non-functional. Each row states preconditions and steps, the expected result, and a priority derived from the requirement's priority in the SRS.

TC ID	Req(s)	Objective	Preconditions / Steps	Expected Result	Priority
TC-01	FR-1	Verify a registered user can log in with valid credentials and is routed to the correct role dashboard. Type: Functional	Pre: User account exists and is active (role = Bank Officer). 1. Navigate to the login screen. 2. Enter a valid user ID and password/PIN. 3. Submit the login form.	User is authenticated and redirected to the Bank Officer dashboard; session is created.	High
TC-02	FR-1, FR-2	Verify the account is locked after three consecutive failed login attempts and access remains blocked until an Administrator unlocks it. Type: Functional	Pre: User account exists and is active. 1. Submit login with an incorrect password 3 times in a row. 2. Attempt to log in again with the correct password.	After the 3rd failed attempt the account is locked; step 2 is rejected with a message to contact an Administrator, even though the password is correct.	High
TC-03	FR-3	Verify a Bank Officer can create a new customer record with valid mandatory details. Type: Functional	Pre: Bank Officer is logged in. 1. Open 'New Customer' form. 2. Enter valid name, DOB, address, phone number and ID reference. 3. Submit the form.	Customer record is created with a unique customer identifier and is visible in the customer list.	High
TC-04	FR-3	Verify the system rejects a customer record submitted with a missing mandatory field. Type: Functional	Pre: Bank Officer is logged in. 1. Open 'New Customer' form. 2. Leave the phone number field empty; fill the rest. 3. Submit the form.	Submission is rejected; the phone number field is specifically flagged as invalid/missing. No record is created.	Medium
TC-05	FR-4	Verify an account cannot be opened with an initial deposit below the account type's minimum balance. Type: Functional	Pre: Customer is active; Savings account type has minimum balance = INR 1000. 1. Open 'New Account' form for the customer, select Savings. 2. Enter an initial deposit of INR 500. 3. Submit.	Request is rejected; system displays the required minimum balance (INR 1000). No account is created.	High
TC-06	FR-5	Verify a frozen account rejects deposits, withdrawals and transfers while remaining viewable. Type: Functional	Pre: Active account A1 exists with balance > 0; Administrator freezes A1. 1. Attempt a deposit to A1. 2. Attempt a withdrawal from A1. 3. View A1's balance and history.	Steps 1 and 2 are rejected with a 'frozen account' message and balance is unchanged; step 3 succeeds and shows correct balance/history.	High

TC ID	Req(s)	Objective	Preconditions / Steps	Expected Result	Priority
TC-07	FR-6	Verify a deposit updates the account balance and records a corresponding transaction entry. Type: Functional	Pre: Active account A1, balance = INR 5000. 1. Deposit INR 2000 into A1. 2. Confirm the transaction. 3. View A1's transaction history.	Balance becomes INR 7000; a new 'Deposit' entry appears with correct amount, timestamp, resulting balance and operator ID.	High
TC-08	FR-7	Verify a withdrawal that would take the balance below the account's minimum balance is rejected and the balance is left unchanged. Type: Functional	Pre: Active Savings account A1, balance = INR 1500, minimum balance = INR 1000. 1. Attempt to withdraw INR 800 from A1. 2. Submit.	Transaction is rejected with a message stating the minimum-balance violation; A1's balance remains INR 1500.	High
TC-09	FR-8, NFR-3	Verify a fund transfer between two active accounts is applied atomically and generates two linked transaction entries. Type: Functional	Pre: Active accounts A1 (balance 5000) and A2 (balance 2000), different customers. 1. Transfer INR 1000 from A1 to A2. 2. Confirm the transaction. 3. Inspect transaction history of both accounts.	A1 = 4000, A2 = 3000; both accounts show a transaction entry with a shared transfer reference; if any step of the operation were forced to fail, neither balance would change.	High
TC-10	FR-8	Verify a transfer is rejected when the source and destination account are the same. Type: Functional	Pre: Active account A1 exists. 1. Initiate a transfer from A1 to A1. 2. Submit.	Transfer request is rejected before any balance is modified; an appropriate error message is shown.	Medium
TC-11	FR-9, FR-10	Verify a Customer can view balance and transaction history only for accounts they own. Type: Functional	Pre: Customer C1 owns account A1; account A2 belongs to another customer. 1. Log in as C1. 2. View balance/history of A1. 3. Attempt to view balance/history of A2 (e.g. by direct account lookup).	Step 2 succeeds and shows correct data in reverse-chronological order. Step 3 is denied and the access attempt is recorded.	High
TC-12	FR-11	Verify interest is not credited twice for the same account and period. Type: Functional	Pre: Interest-bearing account A1; interest run for Sept-2026 already completed. 1. Administrator re-triggers the interest run for Sept-2026.	Run is rejected/skipped for A1 with an 'already processed' message; no duplicate interest transaction is created.	Medium
TC-13	FR-13	Verify a deactivated Bank Officer account can no longer log in, while past transactions still show that officer's ID.	Pre: Bank Officer B1 has processed at least one past transaction. 1. Administrator deactivates B1. 2. Attempt to log in as B1.	Step 2 is rejected. Step 3 still shows B1 as the operator on the historical record.	Medium

TC ID	Req(s)	Objective	Preconditions / Steps	Expected Result	Priority
		Type: Functional	3. View a transaction previously processed by B1.		
TC-14	NFR-1	Verify a single balance-affecting transaction, including persistence, completes within 2 seconds under normal load. Type: Non-Functional (Performance)	Pre: Dataset of ~2000 accounts loaded; normal system load. 1. Perform a deposit transaction. 2. Measure elapsed time from submission to confirmation, including DB write.	Transaction completes and is persisted within 2 seconds.	Medium
TC-15	NFR-4	Verify passwords/PINs are never stored, logged or transmitted in plain text. Type: Non-Functional (Security)	Pre: A user account has been created with a known password. 1. Inspect the database table storing credentials. 2. Inspect application/server logs after a login attempt. 3. Capture network traffic during login (if HTTPS is disabled in a test environment).	The stored value is a salted hash, not the plain-text password; logs never contain the raw password; traffic capture shows no plain-text credential over HTTPS.	High
TC-16	NFR-5	Verify every balance-changing operation is captured in an append-only audit trail that cannot be edited or deleted via the application. Type: Non-Functional (Security / Audit)	Pre: At least one deposit and one transfer have been performed. 1. Open the audit trail as Administrator. 2. Attempt to edit or delete an existing audit entry through the application UI/API.	Audit trail lists timestamp, accounts affected, amount and operator for each entry; the edit/delete attempt in step 2 is refused by the application.	High
TC-17	NFR-7	Verify invalid input (non-numeric / negative amount, unknown account number) is handled gracefully without crashing or altering stored data. Type: Non-Functional (Reliability)	Pre: Active account A1, known balance. 1. Attempt a withdrawal with a negative amount. 2. Attempt a deposit with a non-numeric amount. 3. Attempt a transaction on a non-existent account number.	Each attempt is rejected with a clear error message; the application does not crash; A1's balance is unchanged after steps 1-2; no record is created for step 3.	Medium